Optima User Documentation

PlussMa Contributors

Apr 09, 2026

CONTENTS

1 Optima Algorithms **3**
- 1.1 Public Entry Point . . . 3
- 1.2 Problem Model . . . 3
- 1.3 Shared Runtime Contract . . . 4
- 1.4 Common Outputs . . . 4
- 1.5 Progress Hooks . . . 5
- 1.6 Internal Protocol Presets . . . 6
- 1.7 Stall Taxonomy . . . 6
- 1.8 Example Demos . . . 7

2 Optimizer Surfaces: Functional vs OOP **9**
- 2.1 Functional Surface (`PlussMa`) . . . 9
- 2.2 OOP Surface (`PlussAI`) . . . 9
- 2.3 Practical Rule . . . 10

3 Optima Sequential QP **11**
- 3.1 Mathematical Problem . . . 11
- 3.2 Solver Shape . . . 11
- 3.3 Local Quadratic Subproblem . . . 12
- 3.4 Main State . . . 12
- 3.5 Option Surface . . . 12
- 3.6 Hessian Handling . . . 13
- 3.7 Linear-System Policy . . . 13
- 3.8 Step Acceptance . . . 14
- 3.9 Second-Order Correction . . . 14
- 3.10 Restoration . . . 14
- 3.11 Outputs . . . 14
- 3.12 Example Trace . . . 15
- 3.13 Reading the Iteration Trace . . . 15

4 Optima Internal Protocol **17**
- 4.1 Mathematical Problem . . . 17
- 4.2 Solver Shape . . . 17
- 4.3 Primal-Dual System . . . 18
- 4.4 Main State . . . 18
- 4.5 Barrier Objective View . . . 19
- 4.6 Option Surface . . . 19
- 4.7 Hessian Modes . . . 20
- 4.8 Linear-System Policy . . . 20
- 4.9 Barrier and Complementarity Trace . . . 21

4.10 Feasibility Phase and Recovery 21
4.11 Outputs 21
4.12 Example Trace 22
4.13 Dedicated Demo 22

This Sphinx set is for optimization algorithms owned by `PlussMa`.

Module boundary:

- `PlussQit` owns `Series`, autodiff, and backend compute
- `PlussMa` owns numerical optimization logic
- `optima(...)` is the public constrained-optimization entry point

CHAPTER

ONE

OPTIMA ALGORITHMS

Optima is the constrained optimization layer built in PlussMa on top of the Series and autodiff runtime from PlussQit.

This split is intentional:

- PlussQit owns Series, autodiff transforms, and backend compute
- PlussMa owns numerical optimization logic and solver policy
- optima(...) is the public constrained-solver entry point

1.1 Public Entry Point

The top-level call takes a constrained problem, a ParameterPack<Series> initial point, and an OptimaOptions value:

```
using namespace pluss::subsp::math::optim;

ConstrainedProblem problem;
ParameterPack<Series> x0;
OptimaOptions options;

options.algorithm = OptimaAlgorithm::SequentialQP;
const auto result = optima(problem, x0, options);
```

1.2 Problem Model

Optima works with:

- a scalar objective
- nonlinear equality constraints
- nonlinear inequality constraints
- linear equality constraints
- linear inequality constraints
- lower and upper bounds

The public parameter surface is always ParameterPack<Series>. Derivatives come from either:

- autodiff callbacks over ParameterPack<Trace>
- user-supplied analytic callbacks

Autodiff/analytic are the derivative paths in `Optima`.

1.3 Shared Runtime Contract

All current backends reuse the same evaluation layer and diagnostics:

- callback validation and shape checks
- non-finite evaluation detection
- best-feasible tracking
- multiplier reporting
- progress hooks
- stall classification
- scaling and normalization
- dense and iterative linear-system policies

`OptimaAlgorithm` currently exposes:

- `MeritMethod` (baseline penalty-method backend)
- `SequentialQP`
- `WorkingSet`
- `InternalProtocol`

The primary MATLAB-inspired constrained NLP backends are `SequentialQP` and `InternalProtocol`. This document focuses on those two:

- *Sequential QP*
- *Internal Protocol*

1.4 Common Outputs

Every `optima(...)` result exposes:

- final parameters
- final evaluation bundle
- iteration count
- function evaluation count
- backend parameter
- stationarity metric
- exit flag
- message
- `OptimaOutput`
- `ConstraintMultipliers`
- optional best-feasible snapshot

The `OptimaOutput` payload is the main reporting surface:

Field	Meaning
`algorithm`	Backend label
`iterations`	Major iterations
`function_evaluations`	Total evaluation count
`constraint_violation`	Max violation at the returned point
`first_order_optimality`	First-order stationarity metric
`step_size`	Final accepted step fraction
`line_search_fraction`	Accepted line-search scaling
`backend_parameter`	Merit weight or barrier parameter
`complementarity`	Complementarity residual where applicable
`krylov_iterations`	Iterative linear-solver work
`best_feasible_available`	Whether a feasible checkpoint was found
`returned_best_feasible`	Whether the solver returned that checkpoint
`best_feasible_value`	Objective at the best feasible checkpoint
`best_feasible_constraint_violation`	Violation at the best feasible checkpoint
`best_feasible_first_order_optimality`	Stationarity at the best feasible checkpoint
`multiplier_max_norm`	Largest multiplier magnitude
`active_multiplier_count`	Count of active multipliers
`detail_message`	Expanded termination message
`stall_category`	Structured stall classification
`stall_measure`	Diagnostic measure associated with the stall
`stall_reference`	Reference threshold associated with the stall

1.5 Progress Hooks

Each backend option block accepts a `progress_hook`:

```
options.sequential_qp.progress_hook = [](const OptimaProgressSnapshot& s) {
  return false;
};
```

The snapshot includes:

- `stage`
- `algorithm`
- `params`
- `evaluation`
- `iteration`
- `function_evaluations`
- `step_size`
- `backend_parameter`
- `stationarity`
- `complementarity`
- `krylov_iterations`
- `message`

- stall_category
- stall_measure
- stall_reference
- best_feasible

Returning true requests an orderly stop.

reports::iter(std::ostream&, precision) and reports::iteration_only_trace(...) are convenience factories for per-iteration output.

For a MATLAB OutputFcn-style intercept, use a custom hook and return true when you want to stop before the next iteration.

1.6 Internal Protocol Presets

Both presets below use OptimaAlgorithm::InternalProtocol; the robust preset just allocates more recovery budget and explicit solver defaults:

- presets::internal_protocol()
- presets::robust_internal_protocol()

Key robust deltas:

- max_iterations: 160 (vs 100)
- max_restoration_iterations: 10
- max_restoration_line_search_iterations: 20
- enable_restoration: true
- enable_feasibility_phase: false
- hessian_approximation: Bfgs
- linear_system_solver: Auto
- system_structure: Auto

1.7 Stall Taxonomy

Optima surfaces structured failure modes instead of only generic stop messages:

- none
- user-stop
- invalid-callback-contract
- nonfinite-callback-region
- search-direction-collapse
- boundary-limited
- infeasible-stationary
- complementarity-stagnation
- feasibility-failure
- recovery-failure

- iteration-limit

1.8 Example Demos

Current demos relevant to these backends:

- PlussMa/examples/Numerical/optimization/optima_sequential_qp_demo.cpp
- PlussMa/examples/Numerical/optimization/optima_internal_protocol_demo.cpp
- PlussMa/examples/Numerical/optimization/garch11_optima_demo.cpp
- PlussMa/examples/Numerical/optimization/garch11_internal_protocol_demo.cpp

CHAPTER

TWO

OPTIMIZER SURFACES: FUNCTIONAL VS OOP

There are two ADAM surfaces in the Pluss stack by design.

2.1 Functional Surface (`PlussMa`)

Namespace:

- `pluss::optim::adam_state_like(...)`
- `pluss::optim::adam_step_inplace(...)`
- `pluss::optim::adam_step(...)`

Intent:

- low-level `Series` update primitive
- explicit sensitivity/gradient updates
- useful for custom ML loops and research code

Demo:

- `PlussMa/examples/Numerical/optimization/adam_functional_demo.cpp`
- Includes both explicit-gradient and autodiff-generated-gradient usage.

2.2 OOP Surface (`PlussAI`)

Namespace:

- `pluss::nn::optim::Adam`

Intent:

- module/parameter-list optimizer object
- DL-style training loop ergonomics
- parameter groups and richer optimizer policy

Demo:

- `PlussAI/examples/adam_oop_demo.cpp`
- Includes both explicit-gradient `step(grads)` and autodiff `step()` usage.

2.3 Practical Rule

Use the namespace to communicate intent clearly:

- choose `pluss::optim::*` for functional/math-level optimization utilities
- choose `pluss::nn::optim::*` for neural-module training optimizers

CHAPTER

THREE

OPTIMA SEQUENTIAL QP

`SequentialQP` is the dense constrained nonlinear-programming backend for `optima(...)`.

It is designed for derivative-based constrained optimization over moderate parameter counts, with gradients and Jacobians coming from `PlussQit` autodiff or analytic callbacks.

3.1 Mathematical Problem

The backend targets nonlinear programs of the form

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{s.t.} \quad & c_{\text{eq}}(x) = 0, \\ & c_{\text{ineq}}(x) \le 0, \\ & A_{\text{eq}}x = b_{\text{eq}}, \\ & A_{\text{ineq}}x \le b_{\text{ineq}}, \\ & \ell \le x \le u. \end{aligned}$$

At a candidate solution, the first-order KKT conditions are expressed in terms of the Lagrangian

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \lambda_{\text{eq}}^{\top} c_{\text{eq}}(x) + \lambda_{\text{ineq}}^{\top} c_{\text{ineq}}(x) + \nu_{\text{eq}}^{\top}(A_{\text{eq}}x - b_{\text{eq}}) + \nu_{\text{ineq}}^{\top}(A_{\text{ineq}}x - b_{\text{ineq}}).$$

`SequentialQP` tracks a practical KKT residual derived from the gradient of this Lagrangian together with active constraint residuals.

3.2 Solver Shape

The backend works by repeating this major loop:

1. evaluate objective and constraints
2. linearize constraints at the current point
3. build a local quadratic model of the Lagrangian
4. solve a KKT-style subproblem
5. apply line search and bound-feasible step control
6. optionally run second-order correction
7. optionally run restoration if the main step fails
8. update the Hessian model and multiplier estimate

3.3 Local Quadratic Subproblem

At iteration k, the solver forms a quadratic model around the current point x_k:

$$\begin{aligned} \min_{p} \quad & \nabla f(x_k)^\top p + \tfrac{1}{2} p^\top B_k p \\ \text{s.t.} \quad & c_{\text{eq}}(x_k) + J_{\text{eq}}(x_k) p = 0, \\ & c_{\text{ineq}}(x_k) + J_{\text{ineq}}(x_k) p \le 0, \end{aligned}$$

where:

- B_k is the current Hessian approximation to the Lagrangian
- J_{eq} and J_{ineq} are Jacobians assembled from autodiff or analytic callbacks

The implementation solves the corresponding KKT system using the configured linear-system policy.

3.4 Main State

The internal SQP state tracks:

- current parameters
- current evaluation
- iteration count
- function evaluation count
- current step size
- current penalty parameter
- current merit value
- KKT residual
- convergence flag
- message
- best-feasible snapshot
- Hessian approximation
- multiplier estimate
- variable scaling
- normalization statistics
- Krylov iteration count

3.5 Option Surface

`SequentialQPOptions` exposes:

Option	Meaning
`max_iterations`	Major-iteration budget
`max_line_search_iterations`	Backtracking budget per major step
`max_second_order_corrections`	Second-order correction attempts

continues on next page

Table 1 – continued from previous page

Option	Meaning
`max_restoration_iterations`	Restoration-phase major iterations
`max_restoration_line_search_iterations`	Backtracking budget inside restoration
`initial_step_size`	Starting trial step fraction
`backtracking`	Multiplicative line-search shrink
`armijo`	Sufficient decrease coefficient
`initial_penalty`	Starting merit penalty
`constraint_tolerance`	Feasibility tolerance
`optimality_tolerance`	First-order stop threshold
`step_tolerance`	Small-step stop threshold
`value_progress_tolerance`	Small-objective-change stop threshold
`active_set_tolerance`	Constraint activity threshold
`regularization`	KKT diagonal regularization
`bfgs_curvature_tolerance`	BFGS skip threshold
`second_order_correction_tolerance`	SOC activation threshold
`fraction_to_boundary`	Bound-feasible step fraction
`pivot_threshold`	Pivoted-direct linear solve threshold
`max_krylov_iterations`	Iterative solve budget
`iterative_dimension_threshold`	Size threshold for iterative routing
`krylov_tolerance`	Relative Krylov stop
`krylov_absolute_tolerance`	Absolute Krylov stop
`objective_limit`	Early exit target
`project_bounds`	Bound projection toggle
`enable_restoration`	Restoration fallback toggle
`system_structure`	Dense or sparse hint
`linear_system_solver`	Linear solve policy
`hessian_approximation`	Hessian model policy
`problem_scaling`	Scaling and normalization policy
`progress_hook`	Iteration callback

3.6 Hessian Handling

Current SQP Hessian policies:

- `Bfgs`
- `Identity`

The Hessian model is applied to the Lagrangian approximation inside the QP step.

When `Bfgs` is active, the update is based on the standard secant pair

$$s_k = x_{k+1} - x_k, \qquad y_k = \nabla\phi_{k+1} - \nabla\phi_k,$$

where \$\phi\$ is the merit-gradient model used by the solver. The update is skipped when curvature is too small, controlled by `bfgs_curvature_tolerance`.

3.7 Linear-System Policy

The backend supports a policy-driven solve path:

- `Auto`
- `Cholesky`

- `PivotedDirect`
- `Inverse`
- `PseudoInverse`
- `ConjugateGradient`

`Auto` routes based on dimension and structure hints. Small dense systems usually stay on direct solves. Larger or structured problems can route to Krylov iterations when configured.

3.8 Step Acceptance

`SequentialQP` uses a merit-based acceptance policy with additional constrained safeguards:

- Armijo decrease on the merit model
- stricter handling when the iterate is still infeasible
- second-order correction when linearized feasibility is too optimistic
- boundary-limited capping
- restoration fallback when the main step cannot improve the merit or feasibility model

The main merit model is an exact-penalty style scalarization:

$$\Phi(x; \rho) = f(x) + \rho \, \|v(x)\|_1,$$

where $v(x)$ denotes the stacked constraint residual used by the implementation. The penalty parameter ρ is adapted when feasibility reduction is lagging.

3.9 Second-Order Correction

After a first QP step p_k is found, the solver can attempt a second-order correction when nonlinear residuals are not captured well by the first linearization. In practice this means re-solving with a corrected residual target instead of accepting a step that only looks feasible in the local linear model.

3.10 Restoration

When the main SQP step fails repeatedly, the backend can switch to a restoration phase that prioritizes violation reduction. A successful restoration step resets the local Hessian model before returning to the main loop.

The restoration target is conceptually

$$\min_x \ \|v(x)\|,$$

subject to the same bound handling logic, so that the solver can recover a usable feasible or near-feasible point before resuming the main SQP loop.

3.11 Outputs

The top-level `optima(...)` call returns a state object and a structured `OptimaOutput` payload. For `SequentialQP`, the most important output fields are:

Output field	Meaning
`iterations`	Major SQP iterations
`function_evaluations`	Objective/constraint evaluation count
`constraint_violation`	Maximum final constraint violation
`first_order_optimality`	Final KKT-style stationarity measure
`step_size`	Final accepted step fraction
`line_search_fraction`	Same accepted line-search fraction reported explicitly
`backend_parameter`	Current penalty parameter
`krylov_iterations`	Krylov work if iterative linear solves were used
`best_feasible_available`	Whether a feasible checkpoint was found
`returned_best_feasible`	Whether the solver returned that checkpoint instead of the final iterate
`multiplier_max_norm`	Largest multiplier magnitude in the reported multiplier structure
`active_multiplier_count`	Number of active multipliers
`detail_message`	Expanded exit detail
`stall_category`	Structured stop reason

The returned multiplier structure contains:

- `linear_equalities`
- `nonlinear_equalities`
- `linear_inequalities`
- `nonlinear_inequalities`
- `lower_bounds`
- `upper_bounds`

3.12 Example Trace

A typical progress line from an instrumented SQP run looks like:

```
iter=12 evals=50 objective=1.125000 viol=0.000000 stationarity=0.000000 backend=10.
↪000000 step=1.000000 stall=none
```

Interpretation:

- `value`: current objective
- `viol`: current max constraint violation
- `stat`: KKT-style first-order measure
- `step`: accepted step fraction
- `penalty`: current merit penalty parameter

3.13 Reading the Iteration Trace

A useful progress hook for SQP usually prints:

- iteration index
- objective value
- max constraint violation

- KKT residual or stationarity
- merit or penalty parameter
- accepted step size
- Krylov iterations
- stall category

```
options.algorithm = OptimaAlgorithm::SequentialQP;
options.sequential_qp.progress_hook = [](const OptimaProgressSnapshot& s) {
  std::cout << s.iteration
            << " f=" << s.evaluation.value.item<double>()
            << " viol=" << s.evaluation.max_constraint_violation
            << " stat=" << s.stationarity
            << " step=" << s.step_size
            << " penalty=" << s.backend_parameter
            << '\n';
  return false;
};
```

CHAPTER

FOUR

OPTIMA INTERNAL PROTOCOL

`InternalProtocol` is the Pluss-native public label for the primal-dual barrier backend currently implemented under the `OptimaAlgorithm::InternalProtocol` enum branch.

The intent is straightforward:

- the algorithmic family is still internal-protocol
- the public product naming stays Pluss-native

4.1 Mathematical Problem

The backend targets the same constrained nonlinear program:

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{s.t.} \quad & c_{\text{eq}}(x) = 0, \\ & c_{\text{ineq}}(x) \le 0, \\ & A_{\text{eq}}x = b_{\text{eq}}, \\ & A_{\text{ineq}}x \le b_{\text{ineq}}, \\ & \ell \le x \le u. \end{aligned}$$

For inequality handling, the solver introduces slack variables

$$c_{\text{ineq}}(x) + s = 0, \qquad s > 0,$$

and dual variables $z > 0$ for the slack equations.

4.2 Solver Shape

This backend uses a primal-dual barrier formulation with:

- explicit inequality slacks
- explicit inequality duals
- complementarity tracking
- barrier reduction over outer progress
- feasibility-phase fallback
- restoration handling

The major loop is:

1. evaluate the objective and constraints
2. assemble the barrier or primal-dual stationarity model
3. form a Newton-like or quasi-Newton direction
4. cap the step to preserve interiority
5. apply line search
6. update slack, dual, barrier, and curvature state
7. switch to feasibility or restoration modes if the main step stalls

4.3 Primal-Dual System

The implementation follows a primal-dual barrier structure built around:

$$\nabla_x \mathcal{L}(x, \lambda, z) = 0,$$

$$c_{\mathrm{eq}}(x) = 0, \qquad c_{\mathrm{ineq}}(x) + s = 0,$$

$$Sz = \mu \mathbf{1}, \qquad s > 0,\ z > 0,$$

where:

- $S=\operatorname{diag}(s)$
- μ is the barrier parameter

The complementarity residual reported by the solver is derived from the mismatch in the relation $Sz=\mu \mathbf{1}$.

4.4 Main State

The internal state tracks:

- current parameters
- current evaluation
- iteration count
- function evaluation count
- current step size
- current barrier parameter
- current barrier value
- barrier gradient norm
- convergence flag
- message
- best-feasible snapshot
- Hessian model
- variable scaling
- normalization statistics
- Krylov iteration count

- limited-memory history
- inequality slacks
- inequality duals
- primal residual
- complementarity residual
- feasibility-phase status
- feasibility metric

4.5 Barrier Objective View

In addition to the primal-dual residual system, it is useful to read the method as minimizing a barrier model of the form

$$\phi_\mu(x, s) = f(x) - \mu \sum_i \log(s_i) + \tfrac{1}{2}\rho \, \|c_{\mathrm{eq}}(x)\|_2^2,$$

subject to $c_{\mathrm{ineq}}(x)+s=0$. This is also the right mental model for interpreting:

- `backend_parameter` as the current μ
- `complementarity` as the centrality/complementarity error
- `constraint_violation` as primal infeasibility

4.6 Option Surface

The underlying C++ option type is still `InternalProtocolOptions`. It exposes:

Option	Meaning
`max_iterations`	Major-iteration budget
`max_line_search_iterations`	Backtracking budget per main step
`max_restoration_iterations`	Recovery-phase iterations
`max_restoration_line_search_iterations`	Recovery line-search budget
`initial_step_size`	Starting trial step fraction
`backtracking`	Multiplicative line-search shrink
`armijo`	Sufficient decrease coefficient
`initial_barrier_parameter`	Starting barrier parameter
`minimum_barrier_parameter`	Lower barrier floor
`barrier_decay`	Barrier reduction factor
`equality_penalty`	Equality residual weight inside the barrier model
`centrality_fraction`	Centrality target strength
`dual_recentering_fraction`	Dual recentering amount
`feasibility_activation_factor`	Feasibility-phase trigger scale
`constraint_tolerance`	Feasibility threshold
`optimality_tolerance`	First-order stop threshold
`fraction_to_boundary`	Strict interior step cap
`pivot_threshold`	Pivoted-direct threshold
`initialization_margin`	Distance kept from the boundary at initialization
`bfgs_curvature_tolerance`	Curvature threshold for updates
`lbfgs_memory`	L-BFGS history size
`max_krylov_iterations`	Iterative solve budget

continues on next page

Table 1 – continued from previous page

Option	Meaning
`iterative_dimension_threshold`	Size threshold for Krylov routing
`krylov_tolerance`	Relative Krylov stop
`krylov_absolute_tolerance`	Absolute Krylov stop
`hvp_options`	Hessian-vector autodiff controls
`objective_limit`	Early exit target
`project_bounds`	Bound projection toggle
`enable_restoration`	Recovery phase toggle
`enable_feasibility_phase`	Feasibility phase toggle
`system_structure`	Dense or sparse hint
`linear_system_solver`	Linear solve policy
`hessian_approximation`	Curvature policy
`problem_scaling`	Scaling and normalization policy
`progress_hook`	Iteration callback

4.7 Hessian Modes

Supported curvature policies:

- `Bfgs`
- `Identity`
- `Lbfgs`
- `Exact`
- `HessianVector`

`Exact` and `HessianVector` rely on autodiff-capable callbacks.

For `Exact`, the target object is the Hessian of the Lagrangian-like barrier model with respect to the primal variables. For `HessianVector`, the solver applies matrix-free curvature products inside the iterative subproblem path.

4.8 Linear-System Policy

Supported policies:

- `Auto`
- `Cholesky`
- `PivotedDirect`
- `Inverse`
- `PseudoInverse`
- `ConjugateGradient`

For larger structured systems, `ConjugateGradient` is the main matrix-free path and feeds `krylov_iterations` in the final output.

4.9 Barrier and Complementarity Trace

The most useful live fields are:

- `backend_parameter`
- `complementarity`
- `stationarity`
- `constraint_violation`
- `step_size`
- `krylov_iterations`

Those reveal whether the backend is:

- making central progress
- reducing barrier pressure
- stalling in complementarity
- stuck in feasibility or recovery

4.10 Feasibility Phase and Recovery

When the main primal-dual loop stalls, the backend can switch into a feasibility-focused mode. The intent is analogous to minimizing a violation measure

$$\min_x \ \|v(x)\|,$$

before returning to the main primal-dual trajectory.

The recovery path is separate from the feasibility phase:

- feasibility phase: reduce primal infeasibility
- recovery phase: regain a usable interior step when the barrier step itself fails

These show up in the output through:

- `stall_category`
- `detail_message`
- `best_feasible_available`
- `returned_best_feasible`

4.11 Outputs

The most informative final output fields for `InternalProtocol` are:

Output field	Meaning
`iterations`	Major primal-dual iterations
`function_evaluations`	Objective/constraint evaluation count
`constraint_violation`	Maximum primal infeasibility
`first_order_optimality`	Final stationarity measure
`backend_parameter`	Current barrier parameter μ
`complementarity`	Final complementarity residual
`krylov_iterations`	Krylov work from iterative subproblem solves
`best_feasible_available`	Whether a feasible checkpoint was found
`returned_best_feasible`	Whether that checkpoint was returned
`multiplier_max_norm`	Largest reported multiplier magnitude
`active_multiplier_count`	Active multiplier count
`detail_message`	Expanded termination detail
`stall_category`	Structured stop reason

The final multiplier structure is split into:

- `linear_equalities`
- `nonlinear_equalities`
- `linear_inequalities`
- `nonlinear_inequalities`
- `lower_bounds`
- `upper_bounds`

4.12 Example Trace

A typical progress line from an instrumented run looks like:

```
iter=35 evals=189 objective=16.329592 viol=0.000000 stationarity=0.244045 backend=0.
↪010000 step=0.062500 stall=none
```

Interpretation:

- `value`: current objective
- `viol`: max constraint violation
- `stat`: first-order stationarity
- `compl`: complementarity residual
- `barrier`: current barrier parameter
- `step`: accepted step fraction
- `krylov`: inner iterative solver work

4.13 Dedicated Demo

For an isolated walkthrough, use:

- `PlussMa/examples/Numerical/optimization/garch11_internal_protocol_demo.cpp`

That demo prints:

- the full option block
- every major iteration
- barrier parameter
- complementarity
- stall category
- final multiplier summary